Dual-edge sampling and Circuit5

Two edge-triggered registers hold samples from different moments. The Dualedge explanation shows how to select the newest one, then explains the timing limits of that circuit in an FPGA. Circuit5 shows how to infer a mux from a waveform.

Where this fits in your sheet

Entries 87, 89. Day 5.

Entry numbers follow the tracker. Day labels in older filenames refer to when the notes were written; use the entry numbers to find the matching problem.

Pages	What to read
2-4	Why OR loses the newest value; clock-selected mux solution
5-14	FPGA DDR resources, half-cycle timing, glitches and XOR startup
15-17	Circuit5 selector mapping and case statements
18-19	Simulation checks, sources and revision questions

Reviewed against the saved tracker and available code on 7 September 2026. Earlier submission dates inside these notes are historical records.

1. Question map and the direct FPGA answer

Entries 87 and 89 are on Day 5 in the tracker. This file kept the Day 6 title from its original review date. Entry 90 has its own FSM PDF; the table below shows the surrounding problems.

Entry	Problem	Question status	Document
86	Vectorr	No question recorded	Tracker note only
87	Dualedge	Newest edge, FPGA legality, DDR hardware, timing and glitches	This PDF
88	Thermostat	No question recorded	Tracker note only
89	Sim/circuit5	Waveform inference, case(c), X handling and synthesized hardware	This PDF
90	Exams/2014 q3fsm	Three-sample FSM/counting questions	Dedicated Entry 90 PDF
91	Vector4	No question recorded	Tracker note only
92	Count15	No question recorded	Tracker note only
93	Popcount3	No question recorded	Tracker note only

The direct answer

Generic dual-edge state register in FPGA fabric: normally no. **A falling-edge-only register:** yes. **DDR input or output at a device pin:** yes, using the family-specific IDDR/ODDR, DDIO, GPIO, or serializer/deserializer resource. **HDLBits two-register emulation:** functionally useful, but it does not inherit the glitch-free and characterized timing behavior of a real dedicated DDR cell.

For the FPGA question, distinguish what simulates, what synthesizes, which device resources exist, and whether the implemented circuit meets timing.

2. Entry 87 - why the OR merge fails

The official problem asks for a circuit that behaves like a dual-edge-triggered flip-flop. FPGAs normally provide positive-edge or negative-edge flip-flops, not one storage element triggered by both edges. HDLBits therefore asks for an emulation and explicitly permits the possibility of combinational glitches.

Q: Why is assign q = qpos | qneg wrong?

The two registers do not represent independent pieces of a value that should be ORed. They are two historical samples. After a positive edge, qpos is newest and qneg is stale. After a negative edge, qneg is newest and qpos is stale. OR has no concept of 'newest'; an old 1 can dominate a newly captured 0.

Event	d	qpos	qneg	qpos | qneg	Required q
Initial known example	-	0	0	0	-
Positive edge	1	1 (new)	0 (old)	1	1
Negative edge	0	1 (old)	0 (new)	1 - wrong	0

Main lesson
Two storage registers do not automatically merge into the latest value. The output logic must identify which register was updated by the most recent edge.

3. HDLBits functional solution: select by clock level

Immediately after a positive edge, clk is high, so qpos is the register updated by the latest transition. Immediately after a negative edge, clk is low, so qneg is the latest register. A 2-to-1 multiplexer captures exactly that relationship:

```
module top_module (
    input  wire clk,
    input  wire d,
    output wire q
);
    reg qpos;
    reg qneg;

    always @(posedge clk)
        qpos <= d;

    always @(negedge clk)
        qneg <= d;

    assign q = clk ? qpos : qneg;
endmodule
```

Clock level after edge	Most recent transition	Selected register	Output
clk = 1	posedge	qpos	Latest sampled d
clk = 0	negedge	qneg	Latest sampled d

This is the clearest answer for the HDLBits behavioral exercise. It is composed of two legal single-edge registers plus combinational selection. However, the clock is also a data-path select signal. Real clock-to-Q and routing delays mean the output need not behave like a characterized, glitch-free flip-flop output at the exact transition.

Exercise answer versus implementation recommendation
Use this circuit to understand the logic and to pass the stated functional test. For a real high-speed DDR pin, use the target FPGA family's dedicated DDR I/O primitive or generated IP instead.

4. What does 'allowed in an FPGA' actually mean?

A single phrase can mix four separate questions. A design may be legal Verilog, executable in simulation, accepted by a synthesizer, and still be a poor or impossible mapping for a particular FPGA. Check each layer independently.

Construct	Simulation	Generic FPGA synthesis	Practical verdict
always @(posedge clk)	Yes	Yes	Normal rising-edge register
always @(negedge clk)	Yes	Yes	Normal falling-edge register
always @(posedge clk or negedge clk)	Many simulators execute it	Normally no dual-edge register to infer	Do not use as portable FPGA RTL
Two always blocks driving the same q	May simulate with races/X	Multiple sequential drivers	Invalid architecture
Two registers plus MUX/XOR	Yes	Uses ordinary fabric elements	Functional emulation with caveats
IDDR/ODDR or DDIO/GPIO IP	Vendor model	Yes for supported device I/O	Preferred for real DDR pins
One-edge logic at a 2x clock	Yes	Yes when clocking/timing are valid	Common internal alternative

What the vendor documentation says

Intel Quartus diagnostic 13939 states that integrated synthesis cannot generate logic for a dual-edge register in the device. AMD Vivado's generic flip-flop guidance describes registers using a rising *or* falling-edge clock. AMD then documents separate IDDR/ODDR resources for DDR I/O. Together, these sources support the practical rule above.

This is a mapping limitation, not a claim that the simulator cannot observe both events. Passing a behavioral simulation never proves that a physical register primitive exists.

5. Why ordinary FPGA fabric does not infer one dual-edge flip-flop

An FPGA logic element combines programmable combinational logic with a configurable storage element. The storage element has a clock pin configured for one active polarity. The tool can map either a rising-edge RTL register or a falling-edge RTL register to it, but that does not turn the same storage bit into a register with two active clock edges.

RTL intention	Typical physical mapping	Key point
posedge register	One fabric flip-flop with rising active edge	One update opportunity per period
negedge register	One fabric flip-flop with falling active edge or inverted clock polarity	Still one update opportunity per period
both-edge state bit	No generic one-cell mapping	Needs special cell or a multi-element architecture
DDR pin transfer	Dedicated I/O registers/serializer	Architected specifically for both-edge throughput

Why not drive q from two clocked blocks?

```
always @(posedge clk) q <= d;
always @(negedge clk) q <= d;   // A second sequential driver for q
```

These blocks describe two independent pieces of sequential hardware, both trying to drive one net or variable. FPGA synthesis does not merge them into a magic dual-edge cell. Depending on language and tool, the result is a multiple-driver error, unresolved logic, or a circuit unlike the intended flip-flop.

Why not just invert the clock?

Inverting clk can create or select the opposite edge for a separate register, but a register still responds to only one polarity. Fabric-generated or locally routed clocks also add skew and timing complexity. Use dedicated clocking resources, clock enables, or the documented DDR block instead of building a new clock with ordinary LUT logic.

6. DDR is allowed - through dedicated I/O hardware

Double data rate means transferring one item on each clock edge, so the interface moves two items per full clock period. FPGA families commonly provide dedicated input and output DDR structures close to the I/O pad. They are not merely a generic fabric register configured twice; they are purpose-built capture/launch circuits with characterized modes and timing.

Vendor family example	Input DDR	Output DDR	Where/why
AMD/Xilinx 7 Series	IDDR	ODDR	ILOGIC/OLOGIC near the pad
AMD UltraScale/Versal	IDDRE1 or native PHY resources	ODDRE1 or native PHY resources	Family-specific SelectIO/IOL
Intel/Altera families	DDIO IN / GPIO IP	DDIO OUT / GPIO IP	Peripheral I/O path and generated IP

Input and output are different jobs

- An input DDR block samples the external pin on both edges and presents two logical samples to the core.
- An output DDR block accepts two core-side data values and serializes them onto the pin on alternate edges.
- A bidirectional DDR interface also coordinates output-enable timing; the data and tristate paths must use compatible structures.

Why this is better than a fabric emulation

- The resource is located and routed for I/O timing rather than arbitrary LUT routing.
- The implementation tool understands its edge relationship and device-specific timing arcs.
- The primitive exposes documented reset, enable, latency, and edge-alignment modes.
- Generated IP can add delay chains, half-rate conversion, and placement constraints needed by the device.

7. AMD/Xilinx example: IDDR and ODDR

AMD's 7 Series SelectIO guide documents dedicated IDDR registers in ILOGIC. IDDR supports OPPOSITE_EDGE, SAME_EDGE, and SAME_EDGE_PIPELINED modes. The same-edge modes move the falling-edge sample into a rising-edge-oriented interface for easier core logic integration; the pipelined form adds latency so the pair appears together.

IDDR mode	Core-side timing	Tradeoff
OPPOSITE_EDGE	Q1 changes from the rising sample; Q2 from the falling sample	Most direct, but outputs belong to opposite phases
SAME_EDGE	Both are presented on rising edges, with pairing separation	Easier core clocking, specific ordering/latency
SAME_EDGE_PIPELINED	A complete Q1/Q2 pair is presented together	Adds a clock of latency for clean pairing

Illustrative 7 Series IDDR instantiation

```
// 7 Series example only. Check the primitive for your exact FPGA family.
wire rise_sample;
wire fall_sample;

IDDR #(
    .DDR_CLK_EDGE("SAME_EDGE_PIPELINED"),
    .INIT_Q1(1'b0),
    .INIT_Q2(1'b0),
    .SRTYPE("SYNC")
) input_ddr (
    .Q1(rise_sample),
    .Q2(fall_sample),
    .C (clk),
    .CE(1'b1),
    .D (pin_d),
    .R (reset),
    .S (1'b0)
);
```

This is intentionally labeled family-specific. UltraScale and newer families use different primitives such as IDDRE1/ODDRE1 or native PHY resources. Always open the HDL template or libraries guide for the exact part selected in the project.

8. AMD/Xilinx output DDR and forwarded clocks

ODDR launches D1 and D2 on alternate clock phases through a dedicated output structure. Besides DDR data, an ODDR is often used to forward a clock to an output pin because it creates the output waveform in the I/O resource rather than through a LUT-gated data path.

Illustrative 7 Series ODDR instantiation

```
// Sends d_rise on one edge and d_fall on the other at the output pin.
ODDR #(
    .DDR_CLK_EDGE("SAME_EDGE"),
    .INIT(1'b0),
    .SRTYPE("SYNC")
) output_ddr (
    .Q (pin_q),
    .C (clk),
    .CE(1'b1),
    .D1(d_rise),
    .D2(d_fall),
    .R (reset),
    .S (1'b0)
);
```

Port/value	Meaning
D1 = d_rise	Value associated with one active edge
D2 = d_fall	Value associated with the other active edge
CE	Clock enable for the DDR storage
R/S and SRTYPE	Device-supported reset/set behavior and timing
Q	Dedicated I/O-side serialized output

Important boundary
IDDR/ODDR solve I/O capture or launch. They do not give arbitrary internal RTL state a free-standing 'update on both edges' primitive. For internal logic, consume the two samples as a pair in a normal single-edge clock domain whenever possible.

9. Intel/Altera DDIO and timing constraints

Current Intel/Altera GPIO documentation describes full-rate DDIO input logic that captures the pad on rising and falling edges, then passes the two streams into the core or into half-rate conversion. The output path performs the reverse operation. Exact IP names and migration rules depend on family: older devices use ALTDDIO functions, while newer families use GPIO IP and device-specific I/O resources.

Configuration	What the I/O block does	Core-side result
Full-rate DDIO input	Captures at both edges of the full-rate clock	Two sample streams
Half-rate conversion	Adds another DDIO stage	Wider bus at a slower core clock
Full-rate DDIO output	Combines two core streams	Alternating pin data on both edges

Both edges must be constrained

The Altera timing example applies one input delay for the positive edge and another using **-clock_fall** plus **-add_delay**. This is a concrete reminder that a DDR interface has two capture relationships. Constraining only the rising edge leaves half the interface incorrectly modeled or unconstrained.

```
# Conceptual SDC pattern from the vendor timing method.
set_input_delay -clock virtual_clock 0.25 ddio_in_data
set_input_delay -add_delay -clock_fall -clock virtual_clock 0.25 ddio_in_data
```

Use the board interface's real setup/hold numbers rather than copying 0.25 ns. The example value explains the command structure, not a universal constraint.

10. Which architecture should you choose?

Need	Recommended architecture	Why
Pass the HDLBits functional exercise	Two edge registers plus clk-selected MUX	Directly expresses newest-sample selection
Capture a DDR input pin	Device IDDR/DDIO/GPIO input IP	Characterized I/O capture and edge handling
Drive a DDR output pin	Device ODDR/DDIO/GPIO output IP	Characterized I/O launch and serialization
Process two samples in internal logic	Convert to a 2-bit parallel word, process on one edge	Keeps most logic in one clock domain
Update internal state at twice the rate	PLL/MMCM-generated 2x clock plus ordinary registers	Uses supported clock networks and normal STA
Target an ASIC library with a DET cell	Instantiate/infer only per that library flow	Not portable back to generic FPGA fabric

Why a 2x clock is not automatically identical

A 2x clock gives two rising edges per original period, but the exact phase relationship to the original rising and falling edges depends on the PLL/MMCM configuration, duty cycle, and clock routing. It is a strong internal architecture only when the generated clock is explicitly defined, phase-related, and closed by static timing analysis.

Do not use the emulated q as another clock

The MUX/XOR output is combinational logic. Driving downstream clock pins from it would create a local or fabric clock with uncontrolled skew and possible glitches. Treat it as data. If another block needs an event, use a normal clock plus an enable pulse or a supported clocking primitive.

11. Half-cycle timing: the hidden cost of using both edges

A rising-edge register and a falling-edge register driven by the same waveform are phase-related, not automatically asynchronous. A path launched on one edge and captured on the opposite edge often receives only the high or low portion of the period. For a nominal period T and perfect 50% duty cycle, the raw separation is about T/2, before subtracting clock-to-Q, routing, setup time, skew, jitter, and uncertainty.

Clock waveform	Rise-to-fall window	Fall-to-rise window	Risk
50% duty	0.50 T	0.50 T	Both directions equally tight
60% high / 40% low	0.60 T	0.40 T	Fall-to-rise path is shorter
40% high / 60% low	0.40 T	0.60 T	Rise-to-fall path is shorter

Why duty cycle matters

Vendor timing engines represent the actual rising and falling edge positions in the clock waveform. Duty-cycle distortion therefore reduces one of the two half-cycle budgets. AMD also notes that clock latency and uncertainty alter the ideal edge relationship. A design that passes at a perfect 50/50 behavioral clock can fail on real silicon if the shorter phase has insufficient slack.

The XOR form creates real half-cycle paths

In the feedback construction, qpos feeds the equation captured by qneg and qneg feeds the equation captured by qpos. Those are related rise-to-fall and fall-to-rise timing paths. They must not be casually false-pathed: the next calculation genuinely depends on the opposite-edge register arriving in time.

12. Glitches, metastability, reset, and startup

Why the MUX output can briefly be wrong

At a rising edge, clk changes the MUX selection immediately in the logic model, while qpos changes only after the physical flip-flop's clock-to-Q delay. The MUX can therefore expose the previously stored qpos value before qpos settles to the newly sampled d. A similar handoff occurs at the falling edge. The pulse may be very short, but it means the emulation is not a guaranteed glitch-free flip-flop output.

Sampling twice does not remove metastability

Input d must meet setup and hold requirements at both active edges. If d changes too close to either edge, the corresponding storage element can become metastable. A dual-edge scheme doubles the number of sampling instants; it does not synchronize an asynchronous signal. Use an appropriate synchronizer or source-synchronous interface architecture.

Reset must be designed for both phases

If separate rising-edge and falling-edge registers use reset, each register must meet the reset release requirements relative to its own active edge. Vendor DDR primitives define which reset modes and initial values are supported. In a hand-built two-register circuit, asynchronous deassertion can release the two halves at different effective times unless reset synchronization is planned.

Startup values are part of the specification

Many FPGA flows can initialize registers from the configuration image, but support and behavior are device/tool specific and do not create a portable ASIC reset strategy. If the module has no reset port, state the power-up assumption and verify the synthesized target rather than relying on a testbench's convenient initial values.

13. XOR-feedback alternative and its startup caveat

The saved conversation also presented a compact XOR-feedback construction:

```
// Algebraically elegant, but the two registers need known initial values.
reg qpos = 1'b0;
reg qneg = 1'b0;

always @(posedge clk)
    qpos <= d ^ qneg;

always @(negedge clk)
    qneg <= d ^ qpos;

assign q = qpos ^ qneg;
```

Algebra after a positive edge

```
qpos_new = d ^ qneg
q        = qpos_new ^ qneg
         = d ^ qneg ^ qneg
         = d
```

Algebra after a negative edge

```
qneg_new = d ^ qpos
q        = qpos ^ qneg_new
         = qpos ^ d ^ qpos
         = d
```

Why the XOR version needs initialization

In strict four-state simulation, uninitialized qpos and qneg begin as X. Because X XOR anything remains X, the cross-coupled recurrence can stay unknown forever. The no-reset module therefore needs an explicit and target-valid initialization mechanism. The clock-selected MUX form does not have this cross-coupled-X lockup.

Even after initialization, the XOR architecture has two real half-cycle feedback paths and a combinational output. It is clever and synthesizable from common elements in many flows, but it should not be presented as equivalent to a dedicated, characterized DDR I/O primitive.

14. Entry 89 - infer Sim/circuit5 from waveforms

This problem is combinational. The task is not to recognize a familiar schematic first; it is to infer a mapping from two waveforms. The key observation is that **c** changes through selector values while a, b, d, and e act like candidate data words. The output q follows a different candidate for c = 0, 1, 2, and 3, then becomes F for all other c values.

Q: How do I see what the circuit is doing?

Use a three-step waveform method:

- Hold attention on which input changes in an orderly sequence. Here c walks through values.
- For each c value, match q to one of the data inputs. Record the mapping without guessing the circuit name.
- Use the second simulation, where data values differ, to reject coincidences and confirm the mapping.

Selector c	Observed source for q	RTL meaning
0	b	q = b
1	e	q = e
2	a	q = a
3	d	q = d
4 through 15	constant F	q = 4'hf

15. Why case(c), specifically?

Q: Why do we put c inside the case statement?

A case expression is the value that chooses among behaviors. The waveforms show that c chooses which source reaches q, so c is the selector. The other four signals are data inputs. There is nothing special about the letter c; if a different signal selected the paths, that signal would belong inside case(...).

Signal role	Signals	Question answered
Selector/control	c	Which source should drive q now?
Candidate data	a, b, d, e	What 4-bit value is available on that source?
Fallback constant	4'hf	What appears when c is outside 0..3?

Do not confuse input b with hexadecimal digit B

The identifier **b** names a 4-bit input bus. The literal **4'hb** means the constant hexadecimal value B, decimal 11. For example, if b = 4'h2 and c = 0, q must be 4'h2 because the selector chooses the input bus b; it does not output the literal 4'hb.

Complete combinational implementation

```
module top_module (
    input  wire [3:0] a,
    input  wire [3:0] b,
    input  wire [3:0] c,
    input  wire [3:0] d,
    input  wire [3:0] e,
    output reg  [3:0] q
);
    always @(*) begin
        case (c)
            4'd0:    q = b;
            4'd1:    q = e;
            4'd2:    q = a;
            4'd3:    q = d;
            default: q = 4'hf;
        endcase
    end
endmodule
```

Every selector value receives an assignment because the case includes a default. That prevents latch inference. Declaring q as **output reg [3:0] q** is conventional Verilog syntax for a signal assigned procedurally; it does not mean the circuit contains a clocked register.

16. Deeper waveform-inference and case-statement theory

A repeatable inference method

Step	Question	Circuit5 application
1. Classify	Is q edge-aligned or immediate?	Immediate changes indicate combinational behavior
2. Find the sweep	Which input visits orderly values?	c walks through selector values
3. Match candidates	Which source equals q in each interval?	0:b, 1:e, 2:a, 3:d
4. Break coincidences	Could equal data values hide the source?	Use the second waveform with different data
5. Cover the rest	What happens outside named values?	c = 4..15 gives constant F
6. Encode and test	Does every row have an assignment?	Plain case plus default; test all 16 c values

Plain case, casez, and casex

- Plain **case** performs exact four-state matching. It is the right choice because the legal selector constants are exact.
- **casez** treats Z and ? positions as wildcards. Use it only when the encoding intentionally contains don't-care bits.
- **casex** treats X as a wildcard too. In synthesizable control logic it can hide an unknown selector and make simulation look more deterministic than the hardware diagnosis deserves.
- No wildcard case is needed here. If c contains X or Z, none of 0, 1, 2, or 3 matches exactly, so the default drives 4'hf.

What hardware does this synthesize?

The case describes a 4-bit-wide multiplexer with five data choices: b, e, a, d, and constant F. There is no clock and no intentional memory. The keyword **reg** only permits procedural assignment in Verilog; the complete assignment coverage determines that the result is combinational.

17. Verification record and evidence boundary

Dualedge

- Alternating 0 and 1 values were applied before both positive and negative edges.
- The clock-selected MUX output matched d after every tested edge.
- The XOR-feedback version passed when its required known initialization was modeled.
- The original OR merge diverged, including the stale-1 case described in the Q&A.

Sim/circuit5

- The mapping was checked for all 16 possible 4-bit c values.
- c = 0, 1, 2, and 3 selected b, e, a, and d respectively.
- Every selector from 4 through 15 produced 4'hf.

```
PASS: Dualedge MUX/XOR solutions and all 16 Circuit5 selector values passed;
OR bug diverged as expected.
```

What was checked
The local Icarus checks verify the functional RTL and the documented failure cases. They do not prove placement, timing closure, metastability behavior, or vendor primitive support for a selected FPGA. Those claims require the actual device project, vendor libraries, constraints, synthesis, implementation, and timing reports.

Primary sources researched

HDLBits Dualedge - official dual-edge behavior, FPGA limitation, and glitch caveat.

HDLBits Sim/circuit5 - official combinational waveform-inference problem and module declaration.

Intel Quartus diagnostic 13939 - explicit statement that integrated synthesis cannot generate a dual-edge register in the device.

AMD Vivado Synthesis UG901: Flip-Flops, Registers, and Latches - generic registers use rising or falling-edge clocks.

AMD 7 Series SelectIO UG471 - IDDR/ODDR resources and IDDR edge modes.

Altera Agilex 7 GPIO input path - DDIO input captures rising and falling edge data.

Altera DDIO timing example - constraints for positive-edge and negative-edge input timing.

AMD Vivado Constraints UG903: About Clocks - period, waveform, rising/falling edges, duty cycle, latency, and uncertainty.

User's saved Edge conversation 'Explain Verilog Assignment' - Q&A about the OR bug, clock-selected MUX, XOR derivation, waveform inference, and case(c), reviewed locally 2026-09-03.

Local verification: internal/Verification/day6_nonfsm_selfcheck.sv.

18. Final revision checklist

For Dualedge and real FPGA work

- Ask whether the requirement is an HDL exercise, an input DDR pin, an output DDR pin, or arbitrary internal state.
- Do not equate legal simulation syntax with a synthesizable target primitive.
- A normal fabric register may be rising-edge or falling-edge triggered; that is not the same as both edges.
- Use the selected FPGA family's IDDR/ODDR, DDIO/GPIO, or SERDES resources for real DDR I/O.
- Consult the exact device-family HDL template because primitive names, ports, modes, and reset behavior change.
- Constrain both rising and falling I/O relationships and review setup, hold, duty-cycle, skew, jitter, and uncertainty.
- Treat rise-to-fall and fall-to-rise paths as half-cycle paths unless the architecture deliberately retimes them.
- Do not false-path a real opposite-edge dependency merely to hide a timing failure.
- Do not use a combinational emulation output as a clock; keep it as data or generate a clock enable.
- State initialization and reset assumptions, especially for the XOR-feedback form.
- Remember that neither dual-edge sampling nor DDR hardware synchronizes an asynchronous input.

For Circuit5 and waveform puzzles

- For multiple edge-capture registers, ask which value is newest; do not combine histories with OR or AND by intuition.
- Use a truth trace that forces a stale 1 followed by a new 0. This exposes the OR bug immediately.
- For waveform puzzles, identify the orderly changing signal, map each value to q, then confirm with a second data pattern.
- Put the selector in case(...); put the candidate values on the right-hand side assignments.
- Distinguish an identifier such as b from a literal such as 4'hb.
- Prefer plain case for exact selector decoding; do not use casex to make unknowns disappear.
- Assign q on every combinational path, including a default branch.